

Note on Data Encryption

Riccardo Lo Iacono

September 21, 2026

Contents.

1	Introduction to cryptography and codes	1
1.1	Some prerequisites	1
2	Public key cryptography	5
2.1	RSA	5
2.2	Diffie-Hellman	6
2.3	ElGamal	6
2.4	The hard problems underlying public key encryption . .	7
3	Notion of projective geometry	9
3.1	The projective space	9
	References	10

– 1 – Introduction to cryptography and codes.

The notion of cryptography is an old one; it derived from the necessity of hiding the content of a message when communicating through a medium (a channel for instance). In other words, cryptography was born as a way to make communications more secure. Similar to cryptography, stenography is another way to make communications secure, but the procedure behind it are completely different.

These notes will cover exclusively concepts of cryptography, and are meant to provide a broad view of the field. In addition, some code theory will be discussed half-way through these notes.

– 1.1 – Some prerequisites.

Although these notes are meant to give a broad view to the field, some mathematical background is somewhat required. For this reason, the present section will discuss some basic concepts of abstract algebra, as well as some concepts of discrete math.

Here, we introduce the notions of group, abelian group, ring and field; later, section 3, we discuss some concepts of projective geometry since this provide the basic tools for modern cryptography.

– 1.1.1 – Groups, rings and fields.

To give the most accessible idea of what each of groups, rings and fields are, we can think of this as sets on which some operations are defined; in addition, such operations must satisfy some properties.

Formally speaking, given a set G and a binary operation $* : G \rightarrow G^1$, we say that $(G, *)$ forms a group if all of the following hold.

1. There exists an element $\mathbf{1}_G$ such that $a * \mathbf{1}_G = \mathbf{1}_G * a = a, \forall a \in G$. In other words, G must have a neutral element.
2. For all $a \in G$ there must exists a^{-1} such that $a * a^{-1} = a^{-1} * a = \mathbf{1}_G$. Meaning that each element must have an inverse.
3. The operation $*$ must be associative. That is, for all $a, b, c \in G$ it holds $a * (b * c) = (a * b) * c$.

Additionally, if $*$ is commutative, i.e., $a * b = b * a$, we say that $(G, *)$ is an abelian (or commutative) group.

¹Note that the use of $*$ for the binary operation should not fool the reader in to thinking it as the well known multiplication. Instead, it is a placeholder for any well defined operation.

Section 1 – Introduction to cryptography and codes

As for sets, groups may contain some smaller groups, to which we refer to as sub-groups. Formally, given $(G, *)$ a group, we say that $H \subseteq G$ forms a group under $*$ if $1_G \in H$ and $(H, *)$ forms a group.

Beyond the set-theoretic relation, groups and subgroups are connected by a much more fundamental relation, that regarding their size. This result, known in the literature as *Lagrange's theorem for groups*, states that a finite group G , i.e. a group with finitely many elements, can only have subgroups H such that the number of elements in H divides the number of elements in G . We formally state Lagrange's theorem below.

Theorem (Lagrange's theorem for groups): *Let $(G, *)$ be a group, with $|G| < \infty$. Then, any subgroups $(H, *)$ of $(G, *)$ is such that*

$$|H| \mid |G|.$$

Although groups have some nice properties as the whole group-theory shows, more often than not we need something more powerful. To provide for this need, some more advanced structures were developed, the simplest of which is known as *ring*.

To understand rings, we can see them as an extension of groups by an additional operation. More precisely, we say that $R, +, *$ is a ring if

1. $(R, +)$ form an abelian group.
2. $*$ is associative. That is, for all $a, b, c \in R$ it holds $a * (b * c) = (a * b) * c$.
3. $*$ distributes over $+$. In other words, if for all $a, b, c \in R$ we have that $a * (b + c) = a * b + a * c$.

Rings can sometimes satisfy some additional properties, depending on which they take different names. These additional properties are the following.

1. There exists a neutral element for $*$.
2. $*$ is commutative.
3. Each element in R as inverse with respect to $*$.

A ring with all these additional properties is called *field*. In these notes we denote a generic field by either $\mathbb{F}$ or $\mathcal{K}$, while finite fields will be denoted by $\text{GF}(p)$, where p is usually a prime.

– 1.1.2 – Modular arithmetics.

Most of the groups and ring will be using in these notes, rise naturally from the notion of congruence modulo n , where n is an integer. These groups and rings will be denoted by $\mathbb{Z}_n$, or by $\mathbb{Z}_p$ if $n = p$ is prime.

Section 1 – Introduction to cryptography and codes

But what is congruence? Put in simple words, given two integers a, b , we say that a is congruent b modulo n , if we can write a as the sum of b and some multiple of n . That is,

$$a \equiv b \pmod{n} \iff \exists k \in \mathbb{Z} : a = b + kn.$$

Using the above definition, we get

$$\mathbb{Z}_n = \{m \in \mathbb{N} \mid m < n\}.$$

For instance, $\mathbb{Z}_{11} = \{0, 1, \dots, 9, 10\}$. Sometimes when dealing with rings we will write $\mathbb{Z}_n^*$ to mean the group/ring $\mathbb{Z}_n \setminus \{0\}$.

In most of the algorithms we will discuss, we often make use of the gcd between two given integers. Here we first recall what GCD is, and provide two algorithm to compute it.

Let a, b be two integers. We say that an integer d is the GCD of a, b if the following hold:

$$d \mid a \text{ and } d \mid b \tag{1}$$

and for any other integer d' such that Equation (1) holds, we have

$$d' \mid d.$$

In other words, the d is the GCD of a, b if any integer that divides a, b also divides d .

But how do we compute the GCD of two integers? A result in abstract algebra tells us that we can compute d as the difference between some multiple of a and some multiple of b . This result is known as *Bézout identity*. More precisely Bézout identity states that there exists two integers δ and ϵ , such that $d = \gcd(a, b) = a\delta + b\epsilon$. δ and ϵ are called Bézout coefficients.

In practise we compute the GCD by means of the Euclidean algorithm. Two variants exists, a simple one which returns just the GCD, and one which returns both the GCD and the Bézout coefficients. Here we present both, although for our needs the latter is preferred.

Algorithm 1 Euclidean algorithm
Input: Two integers a and b, with $a > b$. Output: The greatest GCD of a and b. Compute the sequence $\{r_i\}_{i \in \mathbb{N}}$ as follow: $r_0 \leftarrow a, r_1 \leftarrow b$ while $r_k \geq 1$ do $r_{k+1} \leftarrow r_{k-1} - q_k r_k$ end return the last computed r_k

Algorithm 2 Euclidean algorithm

Input: Two integers a and b , with $a > b$.

Output: The greatest GCD of a and b , and the Bézout coefficients.

Compute the sequences $\{r_i\}_{i \in \mathbb{N}}$, $\{s_i\}_{i \in \mathbb{N}}$, $\{t_i\}_{i \in \mathbb{N}}$ as follow:

$r_0 \leftarrow a, r_1 \leftarrow b$

$s_0 \leftarrow 1, s_1 \leftarrow 0$

$t_0 \leftarrow 0, t_1 \leftarrow 1$

while $r_k \geq 1$ **do**

$r_{k+1} \leftarrow r_{k-1} - q_i r_k$

$s_{k+1} \leftarrow s_{k-1} - q_i s_k$

$t_{k+1} \leftarrow t_{k-1} - q_i t_k$

end

return the last computed r_k as GCD and s_k, t_k as Bézout coefficients.

– 2 – Public key cryptography.

Until the first years of the '70s cryptography was mainly using a symmetric approach; that is, sender and receiver had the same encryption/decryption key to cypher their messages. But how should the key be choosed in the first place? This and other reasons, thanks to the contribution of Diffie and Hellman, this symmetric schema was soon replaced by what is known as *public key* cryptography.

Let us denote by $\mathcal{A}$ and $\mathcal{B}$ the two entities involved in the communication, and let $\mathcal{E}$ be a malicious entity trying to intercept the communication. Any public key schema follows the same principle:

1. $\mathcal{B}$ chooses a function f such that, apart for $\mathcal{B}$, its inverse is hard to compute.
2. Let m be the message $\mathcal{A}$ wants to send to $\mathcal{B}$; $\mathcal{A}$ computes $c = f(m)$ and transmits c to $\mathcal{B}$.
3. Upon receiving c , $\mathcal{B}$ who knows how to compute f^{-1} , computes $m = f^{-1}(c) = f^{-1}(f(m))$.
4. Even if $\mathcal{E}$ is able to get to c , it can't retrieve m since f^{-1} is hard to compute.

The remainder of this section will focus on two of the most commonly used public key scemas, namely RSA and Diffie-Hellman. In Section 3, we describe how RSA and Diffie-Hellman can be implemented using elliptic curves.

– 2.1 – RSA.

Taking the name from the three researchers who developed it, Rivest, Shamir, and Adleman, RSA is one of the simplest, yet solid²algorithm in cryptography.

The idea behind RSA is the following: we choose two prime numbers p and q of similar size, i.e., their binary representation uses approximately the same numebr of bits, we compute their product n and use this value to cypher our message. In Procedure RSA, we give a more detailed description of the RSA algorithm.

What we are left with is showing that Line 3 of Procedure RSA, does in fact return m . First observe that

$$c^d = (m^e) = m^{ed}.$$

Using Bézout identity, we get

$$m^{ed} = m^{1-\mu\lambda} = m * m^{-\mu\lambda} = m * (m^\lambda)^{-\mu}.$$

²Here by “solid” we refer to the fact that cracking RSA is somewhat unfeasable.

Section 2 – Public key cryptography

Procedure RSA

- 1 $\mathcal{B}$ chooses two prime numbers p and q with similar size, and computes $n = p * q$ and $\lambda = \text{lcm}(p - 1, q - 1)$. Additionally $\mathcal{B}$ chooses ϵ such that $\text{gcd}(\lambda, \epsilon) = 1$. The pair (n, ϵ) represents $\mathcal{B}$'s public key.
- 2 $\mathcal{A}$ cyphers its message m , which is assumed to be an integer, by computing $c = m^\epsilon \pmod{n}$, and sends it to $\mathcal{B}$.
- 3 To retrieve m , $\mathcal{B}$ computes $m = c^d \pmod{n}$, where d is the Bézout coefficient of ϵ in the identity $1 = d\epsilon + \mu\lambda$.
- 4 If $\mathcal{E}$ gets to c , although has access to $\mathcal{B}$'s public key, can't possibly retrieve m . The only possibility is for $\mathcal{E}$ to factor n , which is a well known hard problem.

Recall that by definition $\lambda = \text{lcm}(p - 1, q - 1)$, hence there exists r, s such that $\lambda = (p - 1)r = (q - 1)s$. By using Fermat's theorem, which roughly states that $x^p \equiv x \pmod{p}$, we get

$$m^\lambda = m^{(p-1)r} = (m^{(p-1)})^r = 1^r = 1 \pmod{p}.$$

By the same logic, we get that $m^\lambda = 1 \pmod{q}$. Therefore,

$$c^d = m^{ed} = (m^\lambda)^{-\mu} = m * 1 \pmod{p} = m * 1 \pmod{q} = m.$$

– 2.2 – Diffie-Hellman.

Born by the collaboration between Diffie and Hellman, the Diffie-Hellman key exchange protocol was the first public key exchange algorithm. Roughly speaking the algorithm is the following: we select a prime number p and consider and choose an element $\gamma \in \text{GF}_p$, both parties, in our case $\mathcal{A}$ and $\mathcal{B}$, choose a private key, let these be a for $\mathcal{A}$ and b for $\mathcal{B}$, and compute the public key by taking the a -th (b -th resp.) power of γ .

A more detailed description of the algorithm is shown in Procedure Diffie-Hellman-Key-Exchange.

As stated, Diffie-Hellman is more a public key exchange protocol rather than an encryption schema; though, as described in the next section, when combined with Vernam cyphering algorithm, it allows to build a solid public key encryption schema known as the *ElGamal* encryption schema.

– 2.3 – ElGamal.

As anticipated in Section 2.2, the ElGamal encryption schema is based on the Diffie-Hellman algorithm to exchange the key, what is left is understanding how the message is encrypted/decrypted.

Section 2 – Public key cryptography

Procedure Diffie-Hellman-Key-Exchange
1 Let $\mathcal{T}$ be a trusted third party. $\mathcal{T}$ chooses a prime number p and chooses an element $\gamma \in \text{GF}_p$. 2 Consider $\mathcal{B}$; it chooses an integer b and computes $\beta \equiv \gamma^b \pmod{p}$, which would represent its public key. In the same manner $\mathcal{A}$ computes its public key $\alpha \equiv \gamma^a \pmod{p}$. 3 Both $\mathcal{A}$ and $\mathcal{B}$ can compute the same private key $\kappa_{AB} \equiv \beta^a \pmod{p}$, since $\beta^a = \alpha^b$. 4 Assuming $\mathcal{E}$ intercepts either of α and β, it won't be able to retrieve κ_{AB} since a and b are unknown.

Using the same notation as before, the sender, which we assume to be $\mathcal{A}$, computes $c = \kappa_{AB} * m$ and sends it to $\mathcal{B}$, where m is assumed to be an integer. On the receiving side, $\mathcal{B}$ retrieves m by computing $m = \kappa_{AB}^{-1} * c$. The question is, how is κ_{AB}^{-1} obtained? To answer this question observe that $\text{gcd}(\kappa_{AB}, p) = 1$, meaning that we can use Bézout's identity to find x and y such that $1 = \kappa_{AB}x + py$.

In Procedure ElGamal we give the complete algorithm.

Procedure ElGamal
1 By using Procedure Diffie-Hellman-Key-Exchange, let $\mathcal{A}$ and $\mathcal{B}$ compute their public keys α and β, and the secret key κ_{AB}. 2 Let m be an integer that represents the message $\mathcal{A}$ wants to send to $\mathcal{B}$. $\mathcal{A}$ computes $c = \kappa_{AB} * m$ and sends it to $\mathcal{B}$. 3 Receiving c, $\mathcal{B}$ first computes κ_{AB}^{-1} solving Bézout's identity, then retrieves $m = \kappa_{AB}^{-1} * c$. */ 4 Even if $\mathcal{E}$ obtains c, since it has no access to κ_{AB}, retrieving m is impossible.

– 2.4 – The hard problems underlying public key encryption.

Now that we have seen a couple of public key encryption algorithm, a question should pop inside the reader's head: what guarantees do we have about these algorithms?

It should come to no surprise that the answer is given by math. Precisely, by the hardness of the problems underlying these algorithms. For instance, the security of RSA is given by the hard problem of integers factorization,

Section 2 – Public key cryptography

which asks us, given an integer n , to find its factorization into primes. The Diffie-Hellman algorithm, and therefore ElGamal, are based on the *discrete algorithm* problem, which roughly asks, given an integer α , to find a such that $\alpha \equiv \gamma^a \pmod{p}$, where γ and p are known.

These we presented are just a couple of several public key encryption algorithms. The interested reader can easily find a whole literature out there.

– 3 – Notion of projective geometry.

As anticipated in the introduction, modern days cryptography is based on the notion of elliptic curve. Before we can define what these curves are, we have to understand what a projective space is.

– 3.1 – The projective space.

Projective spaces are deeply rooted in art, as these were implicitly when dealing with perspective. As it turns out, perspective is nothing else but a consequence of how the projective space works.

Formally speaking, an n -dimensional projective space over a field $\mathcal{K}$ is defined as the set of non-empty *rays* $\mathbf{v} \in \mathcal{K}^{n+1}$. More precisely

$$\mathbb{P}^n(\mathcal{K}) = \{P = \langle \mathbf{v} \rangle \mid \mathbf{0} \neq \mathbf{v} \in \mathcal{K}^{n+1}\},$$

where $\mathbf{v} = [x_0, \dots, x_n], x_i \in \mathcal{K}$.

Points in $\mathbb{P}^n(\mathcal{K})$ are represented using two types of coordinates:

1. *homogeneous* coordinates, by which a point Q differs from P if and only if $Q \neq \lambda P$. In other words, using homogeneous coordinates, all points that differ by a multiplicative constant represent the same point.
2. *non-homogeneous* coordinates, by which P is written as

$$P = \begin{cases} [1, x_1/x_0, \dots, x_n/x_0] & \text{if } x_0 \neq 0, \\ [0, x_1, \dots, x_n] & \text{if } x_0 = 0. \end{cases}$$

In the latter case, we say that P is proper (or affine) point, if $x_0 \neq 0$, we say that it's an improper point (or a point at infinity) if $x_0 = 0$.

Our discussion on elliptic curves will involve points in $\mathbb{P}^2(\mathcal{K})$, and we mainly use the non-homogeneous coordinates system. For this reason, to ease the notation a bit, we write $P = [1, x_1/x_0, x_2/x_0]$ as $P = [1, x, y]$, where $x = x_1/x_0$ and $y = x_2/x_0$. For point at infinity we write P as $P = [0, X, Y]$.

– 3.1.1 – homogeneous polynomials.

References.

- [1] W. Diffie and M. Hellman. “New directions in cryptography”. In: *IEEE Transactions on Information Theory* 22.6 (1976), pp. 644–654. DOI: [10.1109/TIT.1976.1055638](https://doi.org/10.1109/TIT.1976.1055638).
- [2] R. L. Rivest, A. Shamir, and L. Adleman. “A Method for Obtaining Digital Signatures and Public-key Cryptosystems”. In: *Commun. ACM* 21.2 (Feb. 1978), pp. 120–126. DOI: [10.1145/359340.359342](https://doi.org/10.1145/359340.359342).